

BACHELOR

Implementing extremum seeking control on a physical testing setup

de Korte, Niels E.

Award date:
2026

[Link to publication](#)

Disclaimer

This document contains a student thesis (bachelor's or master's), as authored by a student at Eindhoven University of Technology. Student theses are made available in the TU/e repository upon obtaining the required degree. The grade received is not published on the document as presented in the repository. The required complexity or quality of research of student theses may vary by program, and the required minimum study period may vary in duration.

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain

Take down policy

If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.



Department of Mechanical Engineering
Control Systems Technology section

Implementing extremum seeking control on a physical testing setup

Niels de Korte (1834932)
Bachelor thesis

Supervisors:

prof. dr. ir. Thijs van Keulen
Juan Javier Palacios Roman

Eindhoven, 23 01 2026

Contents

1	Introduction	1
2	System description	2
2.1	Physical system	2
2.2	Software implementation	2
3	Controller design	4
3.1	General control design	4
3.2	Miscellaneous simulation elements	4
3.3	Classic extremum seeking control	6
3.4	Moving average extremum seeking control	6
3.5	Controller tuning	7
3.6	Simulations	7
3.7	Safe operating range	9
4	Controller testing	10
4.1	Derivative estimator	10
4.2	Extremum seeking testing	11
5	Discussion	13
5.1	Derivative Estimators	13
5.2	General simulation	13
5.3	Motion controller	13
6	Conclusion	14
7	References	15
A	Appendix A	16
B	Manual building an Extremum Seeking Controller	18
B.1	System design	18
B.2	System Analysis	20
B.3	Choosing ESC variables	20
B.4	Common errors	21

1 | Introduction

Extremum seeking control (ESC) is an optimization technique that allows a system to converge to a certain optimum, this can be either a minimum or maximum of an objective function. ESC works by inputting a small perturbation into the system and seeing how the output responds. Using the variation observed in the output, the controller can estimate the local gradient of the objective function. The controller then changes its input to move toward the optimum (either actual movement or optimization). This is very useful in situations where the optimum value or shape of the objective function is unknown. A good example of this is combustion engines: these require very specific timings to run optimal. Using ESC very small changes are made in this timing, then the controller observes the differences in power output and emission and improves the timing slightly. This happens constantly, which causes the system to constantly be moving toward the optimal timings until they are reached.

The experimental setup used for this report consists of a camera mounted on a spindle driven by a motor, constantly measuring the light intensity emitted by a laser. The goal is to get the system to position the camera at the peak of the light intensity. The optimum is not known beforehand. To achieve this goal a controller is needed, which is stable, can constantly estimate the gradient and adjust the camera accordingly.

This report will show the full development process of these controllers, such as system analysis, simulation design and real world tests. Two working controllers are created using different approaches to ESC. Both are then tuned to be able to converge within 20 seconds. They are tested in simulation and compared based on several parameters, such as convergence speed and stability. These simulations are then tested using experimental measurements. Each of these measurements were done multiple times from different starting points for an accurate result. Safe operating ranges were also identified to ensure no unstable controller is used on the setup.

The report is divided into several chapters. Chapter 2 describes the physical setup and the way it communicates with the controller. Chapter 3 elaborates on the theory of ESC and its components. In addition, this chapter includes the difference between both of the used control strategies and the simulation results. In chapter 4 the real world test results are presented and explained. Finally, the appendices include a user manual for designing an extremum seeking controller and a layout of the wiring.

2 | System description

The goal of this project is to design a well functioning extremum seeking controller for the experimental setup. But before starting with the controller design, it is vital to properly understand the setup. The most important parts of the setup will be described here.

2.1 | Physical system

The core of the system consists of a camera that moves along a spindle. This camera is used to measure light intensity coming from the opposite side, where a laser is mounted. This laser emits a light profile toward the camera. A 10W brushless DC-motor is used to rotate the spindle, which converts the torque into a translational force acting on the camera. A linear guide is installed to ensure that all degrees of freedom are constrained, except the translation caused by the motor. On the motor rotor a quadrature encoder is mounted, which provides incremental angle of rotation measurements. The encoder has a resolution of $3.14 * 10^{-3}$ rad, which is equal to 500 slits per revolution. There are also three light gates placed at the right bound, left bound and at the optimum. An image with the numbered components of the setup can be seen in Figure 2.1.

- 1. Spindle
- 2. Camera
- 3. Laser
- 4. DC Motor
- 5. Light gates
- 6. Linear guide

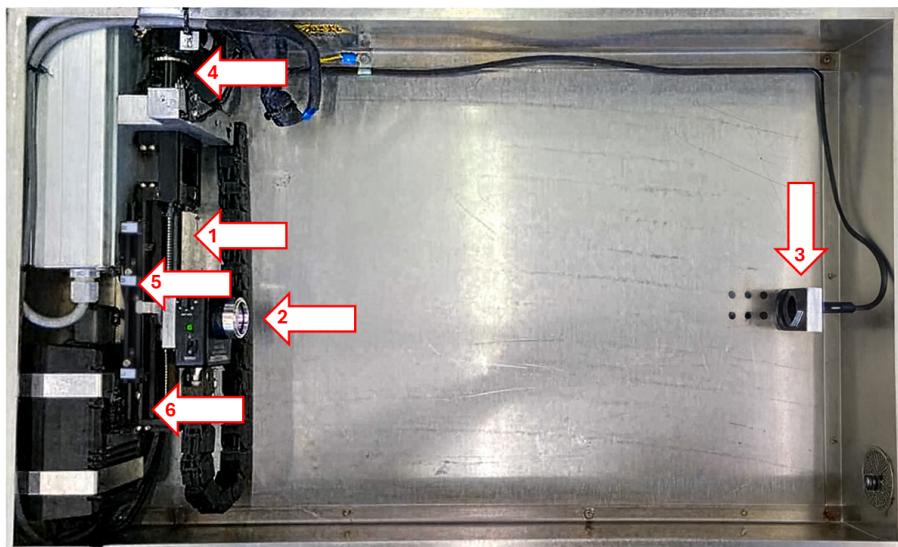


Figure 2.1: Experimental setup

2.2 | Software implementation

All signal processing and communication are handled by an E/box. The E/box receives an analog input from the camera and outputs an analog signal to the motor controller. Also, all encoder signals go through the E/box to ensure accurate position counting. The light gates are used to detect if the camera is going out of bounds. The sampling rate of the E/box is 4KHz, which means the system is in discrete time. This will be essential for the controller later.

A Raspberry Pi 4B is used to run the control algorithm, which is built in simulink and compiled into C code for the hardware. The user is able to start measurements or change parameters while the simulink

model is running via simulink itself, or via Matlab code. The Raspberry Pi and the E/box communicate through an EtherCAT connection. The full wiring can be found in Appendix A. The Simulink design includes the Laser Setup Subsystem, which interfaces with the E/box. This subsystem manages motor control, encoder data, camera measurements and light gate signals. It does this using one input and two outputs. The input is the voltage sent to the motor. This value is calculated by the motion controller. The outputs are the current motor position (rad) and the current light intensity (V) measured by the camera. Measurements are collected using scope blocks, which causes the data to be available in either the scope or the data inspector. This allows for good comparisons between simulation and real life tests. The layout of the laser setup subsystem can be seen in Figure 2.2.

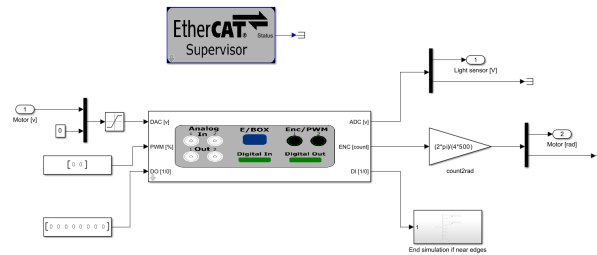


Figure 2.2: Content laser setup block

3 | Controller design

The goal of this controller is to move the camera to the peak in the light intensity graph, thereby achieving the optimization goal. To create such a controller, testing is needed. In order to test the controllers in a quick and accurate way a simulation is designed of the system. How this is done in simulink and Matlab will be discussed in this section.

3.1 | General control design

In a previous research project a controller was already designed that allowed the system to move along trajectories or to move to a certain setpoint [Varnavides \(2025\)](#). Therefore the design of this motion controller will not be discussed here. The transfer function of this controller can be seen in Equation 3.1

$$C_2(s) = (0.6 + 9.64 * 10^{-3}s)(1 + \frac{379}{s})(\frac{s^2 + 377s + 9.87 * 10^6}{s^2 + 333s + 7.71 * 10^6})(\frac{\frac{s}{377} + 1}{\frac{s}{1068} + 1})\frac{1}{2.8 * 10^{-7}s^2 + 7.58 * 10^{-4}s + 1} \quad (3.1)$$

The extremum seeking controller consists of five main parts, the first being a dither signal generator. This provides a sine wave $d(t)$ to the motion controller: $d = a * \sin(\omega t)$, where ω is the dither frequency and a is the dither amplitude. The motion controller then uses this signal to provide a signal to the motor using the laser setup block. This block then gives two measurements from the setup back to the system. The motor position is fed back to the motion controller and the light intensity y moves on to the derivative estimator. Here, using the light intensity signal a local gradient is estimated. This is done by using the dither signal. The dither signal causes the camera to oscillate, which means it can measure the light intensity to the right and left of the current operating point. These light intensities are then compared and an estimate is made for the local gradient. Based on the estimated derivative, the derivative estimator determines what direction to move in order to get closer to the objective point. This derivative estimate signal ξ is then sent to the optimizer, where it is multiplied by a gain value and sent through an integrator. This signal $\hat{u}$ is then added to the first dither signal which provides a signal for the motion controller. This signal then becomes: $u = \hat{u} + d(t)$, where you can clearly see the bias added from the $\hat{u}$, which decides where the camera should move. A general controller scheme can be seen in Figure 3.1.

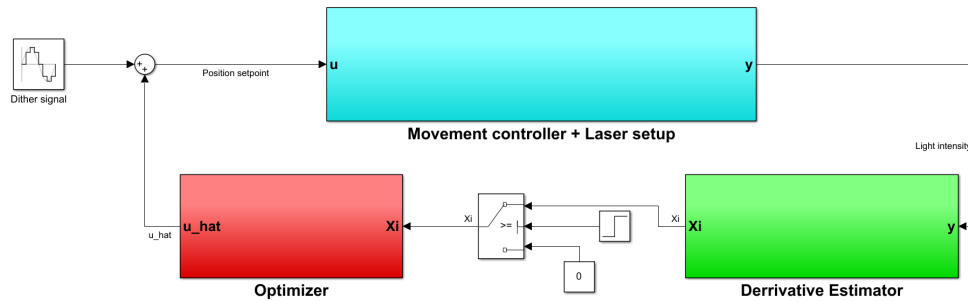


Figure 3.1: General control loop

3.2 | Miscellaneous simulation elements

3.2.1 | Controller limit & Discrete converter

A zero order hold (ZOH) block in simulink is used between the controller and the plant as the controller is a discrete transfer function and the plant is in continuous time. This means that the plant expects a continuous stream of signal updates, but the discrete controller can only give updates after a sampling period is over. For example. the controller only gives out $u(1)$ and $u(2)$, but cannot give out $u(1.5)$. To solve this the zero order hold block takes the value from the controller and keeps it constant at that value until the new value from the controller comes through after one sampling period. So that in this case $u(1.5) = u(1)$ This way the plant gets its continuous information and the system remains stable.

In the actual setup the control output is limited at 2.5 and -2.5 volts. In order to simulate this in the simulation a saturation block is added between the controller and the plant. Even though in most

simulations this value gets nowhere near the maximum, it is still good to have for the accuracy of simulations. The layout of these blocks can be seen in Figure 3.2

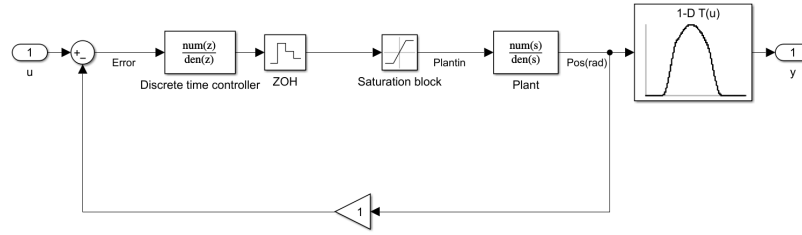


Figure 3.2: Plant layout

3.2.2 | Derivative estimate switch

On the real setup it is very easy to change the starting point of the camera, as it can be moved by hand. In the simulation this is not the case, at the start of the simulation it is assumed to be at zero and then moved to a coordinate entered in by the user. This jump happens in a fraction of a second, which causes issues with the derivative estimator values as it does not follow a normal movement. Additionally during the first period of the dither signal the derivative estimator does not return accurate derivative estimates. To prevent the derivative estimator messing with the early parts of the simulation it is disabled for the first few seconds. This is done with a switch block placed between the derivative estimator and the optimizer (green and red blocks) with default value 0. This is also connected to a step input which changes to a value of 1 after a set time. This then flips the switch which enables the derivative estimator to communicate its value through to the optimizer block. Removal of this section or a derivative estimate switch that is too short, will cause the system to become unstable.

3.2.3 | Objective function implementation

In the simulation it is not possible to get real time data from the sensor to see the light intensity at the point. Instead the measured light intensity can be simulated with two different solutions. The light profile emitted by the laser was measured and plotted in Matlab, this graph can be seen in Figure 3.3. Using a Matlab 'fit' function with 'fitType' 'smoothingspline', this data was converted to 2 variables holding the x and y values. This is then imported from the workspace to the simulink file using a 2-D lookup table. The plant output gets put into this lookup table, which then gives a value for the light intensity similarly to what the sensor would do. The advantage of this is that this objective function is very similar to the actual light profile as it is made from an actual measurement from the setup. The downside is that it can create some numerical difficulties at low amplitudes ($a < 0.01$), which causes the simulation to break. therefore an alternative was made to this lookup table. A gaussian function was manually created to mimic the light profile curve. The best fitting function was found to be Equation 3.2.

$$Q = 0.5627 * e^{\left(\frac{-(u-120)^2}{950}\right)} \quad (3.2)$$

The advantage of this is that this implementation is much easier as it just requires a Matlab function block. It also does not run into the same difficulties at small values of the amplitude as it is a continuous function. However it is a much less accurate representation of reality compared to the lookup table. Simulations were run with both solutions and there was no noticeable difference. All the figures in this report will be made with the lookup table for consistency.

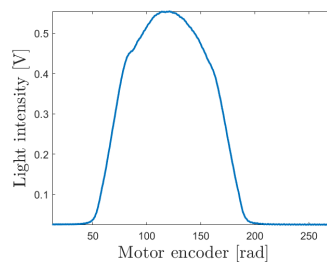


Figure 3.3: Light intensity profile

3.3 | Classic extremum seeking control

For this project two different kinds of derivative estimators are used. The first variant uses a high pass filter, low pass filter, and a demodulation signal. This is commonly referred to as classic extremum seeking control. The signal going into the derivative estimator is $y = Q(u)$, which is the light intensity at coordinate u . This signal when linearized around $\hat{u}$ can be approximated by Equation 3.3:

$$y = Q(\hat{u}) + a * \frac{dQ}{du}(\hat{u}) * \sin(\omega t + \phi) \quad (3.3)$$

Here $Q(\hat{u})$ is the static function evaluated at the operating point (static part), $a * \frac{dQ}{du}(\hat{u}) * \sin(\omega t + \phi)$ is the dither induced oscillation and $\frac{dQ}{du}(\hat{u})$ is the derivative. The first part of the derivative estimator is a high pass filter which filters out the static component. The remaining signal $a * \frac{dQ}{du}(\hat{u}) * \sin(\omega t + \phi)$ is then multiplied with a demodulation signal $\frac{2}{a} \sin(\omega t)$. After some trigonometry this new signal can be rewritten into Equation 3.4:

$$m(t) = \frac{dQ}{du}(\hat{u}) - \frac{dQ}{du}(\hat{u}) * \cos(2\omega t + \phi) \quad (3.4)$$

Now a low pass filter is used to filter out the 2ω (high frequency) component. This then just leaves a signal of $\xi \approx \frac{dQ}{du}(\hat{u})$. Due to the nature of classic extremum seeking control, the derivative estimate will always be the shape of a sine wave. This is caused by the low pass filter not being able to filter out the $\cos(2\omega t + \phi)$ term perfectly, meaning a sine wave gets added to the derivative estimate. In the next section a design is used in which this is not the case. An important part of this derivative estimator is the filter cutoff frequencies. The low pass filter only filters out the unwanted component, when the cutoff frequency is much lower than omega. Therefore a standard value of $\omega/10$ is used. The derivative estimator simulink layout can be seen in Figure 3.4

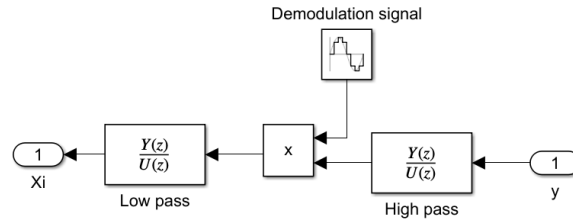


Figure 3.4: Classic derivative estimator

3.4 | Moving average extremum seeking control

The second version of a derivative estimator uses a moving average to remove all the unwanted terms. This version first multiplies with the same demodulation signal as in the previous version. This results in approximately Equation 3.5

$$m(t) = \frac{2}{a} * Q(\hat{u}) * \sin(\omega t) + \frac{dQ}{du}(\hat{u}) - \frac{dQ}{du}(\hat{u}) * \cos(2\omega t + \phi) \quad (3.5)$$

This signal is then put through a moving average, which is defined as: $MA = \frac{1}{T} \int_{t-T}^t m(\tau) d\tau$. This integral will simplify to just the derivative estimate if a window length is taken which is equal to the perturbation period. It can also be reasoned why this approach works. Both unwanted components ($\frac{2}{a} * Q(\hat{u}) * \sin(\omega t)$ and $\frac{dQ}{du}(\hat{u}) * \cos(2\omega t + \phi)$) are terms that oscillate. By taking the average of these over exactly one perturbation, the negative and positive parts will cancel each other out. This will then just leave the constant term which is the gradient estimate. This is why it is vital to take the window length equal to the perturbation period (in samples) of the dither signal, as it otherwise will not cancel out the unwanted components. In this instance a window length of 4000 is used as the sampling rate is 4kHz and the dither frequency is 1Hz. This estimate is then multiplied with a gain value to scale it to the right magnitude. The gain block should have the same value as the dither frequency in Hz (in this case 1). The simulink layout for this design can be seen in Figure 3.5. The advantage of using this version over the previous one, is that it is much simpler and is easier to compute for the simulations.

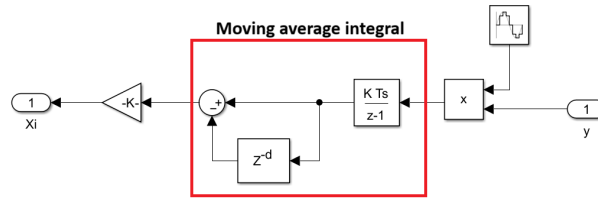


Figure 3.5: Moving average derivative estimator

3.5 | Controller tuning

After making the general structure of the controller, the tuning can begin. For the tuning there are three main values to tune. Those being the dither amplitude, dither frequency and the controller gain. For all three of these values it is the goal to make them as large as possible without making the system unstable. The dither amplitude and frequency are quite self explanatory. The higher frequency means that the system has more measurements to compare which means that it can find the optimum quicker, but a frequency that is too high runs the risk of the motion controller not being able to keep up which makes the system unstable. A higher amplitude means that it takes larger steps which makes it easier for the derivative estimator to find the gradient. This results in a quicker convergence to the optimum. However if the amplitude gets too large it will result in the system overshooting the optimum which leads to irregularities from the derivative estimator, which leads to an unstable system. The same logic applies for the controller gain, this is the gain by which the derivative estimate is multiplied in the optimizer section. A higher gain means that the system moves farther in the estimated direction, but if this gain is too high it will overshoot once again resulting in an unstable system. A full instruction on how to build and tune the controller can be found in the manual (Appendix B)

3.6 | Simulations

3.6.1 | Simulation settings

For this project a frequency of 1Hz was chosen to make the tuning process easier, which means there were only two values to tune, those being the amplitude and the controller gain. An acceptable time for convergence was set at about 20 seconds. The tuning was started with 5% of the total domain as amplitude. For all simulations, the derivative estimator switch was set to 2 seconds. Both simulations started at the same point. The final values used can be seen below and the results can be seen in Figure 3.6 & Figure 3.7.

Classic extremum seeking control

- $f = 1 \text{ Hz}$
- $a = 5 \text{ rad}$
- Gain = 4000

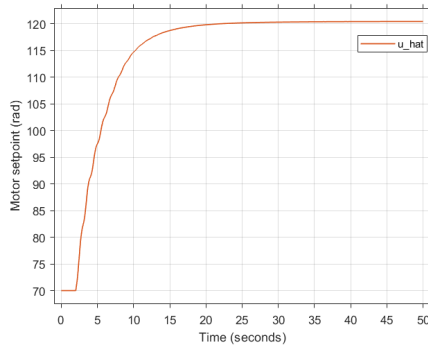
Moving Average extremum seeking control

- $f = 1 \text{ Hz}$
- $a = 5 \text{ rad}$
- Gain = 2500

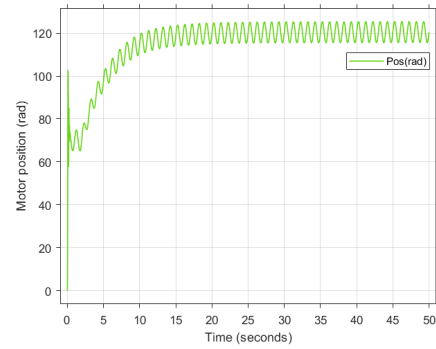
3.6.2 | Simulation results

When analyzing the results, both controllers prove to be effective in finding the optimum. Both are able to converge to the optimum in the set time limit. They both do however behave quite differently. The classic controller shows large oscillations around the optimum and a long settling time. Meaning it moves in the right direction very quickly, but then struggles to stay on the exact optimum. This is caused partly by the high gain, but also by the dynamics of the filtering and the phase lag caused by the filters. The moving average however settles on the exact optimum much quicker, with a smoother trajectory to get there. Once it gets near the optimum it almost instantly settles on it, instead of oscillating around it before settling.

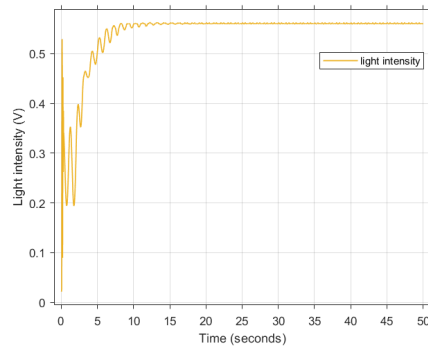
These results prove the theoretical advantages of the moving average method. By averaging the values over exactly one period the oscillation is canceled out, which causes the cleaner gradient estimate and the stability around the optimum. While the classic version can move very aggressively toward the optimum, the oscillating derivative estimate makes it less efficient for the quick and precise positioning.



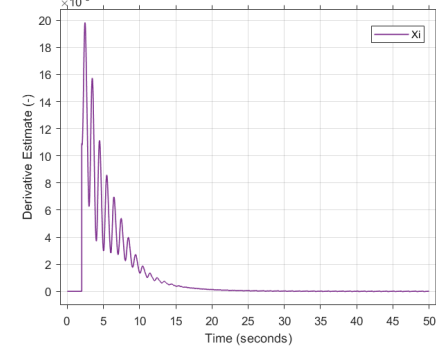
(a) Motor setpoint Classic



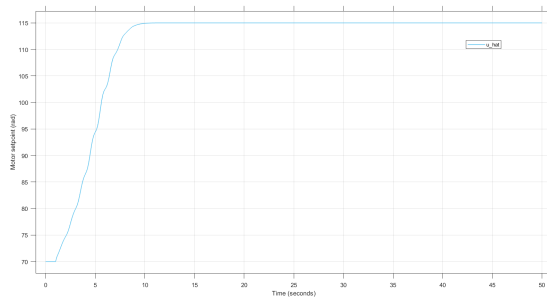
(b) Motor position Classic



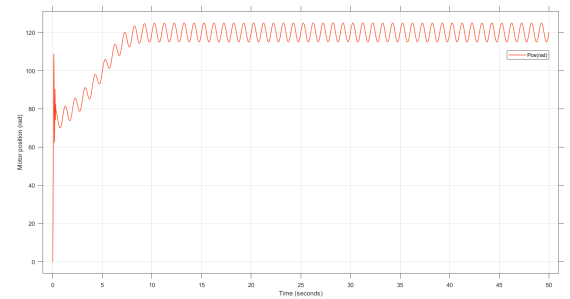
(c) Light intensity Classic



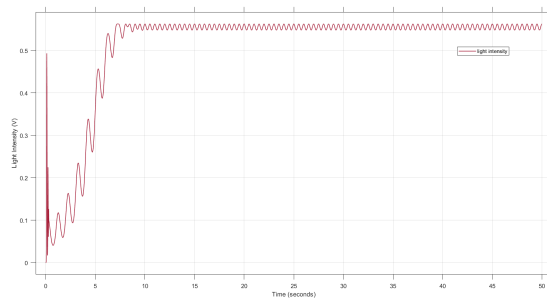
(d) Derivative Estimate Classic

Figure 3.6: Results Classic extremum seeking control simulation

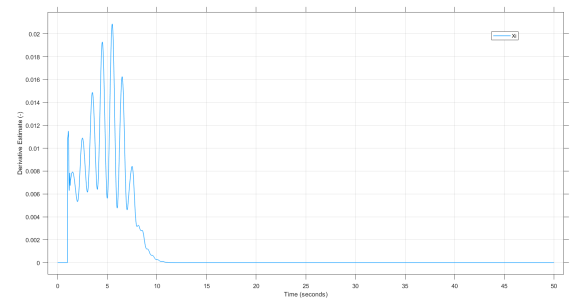
(a) Motor setpoint Moving Average



(b) Motor position Moving Average



(c) Light intensity Moving average

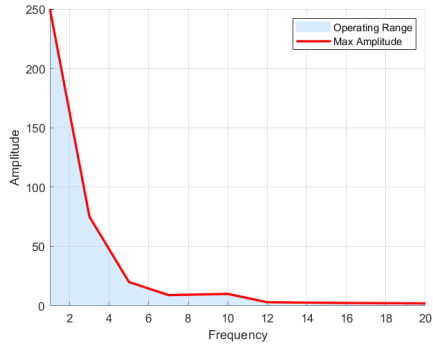


(d) Derivative Estimate Moving Average

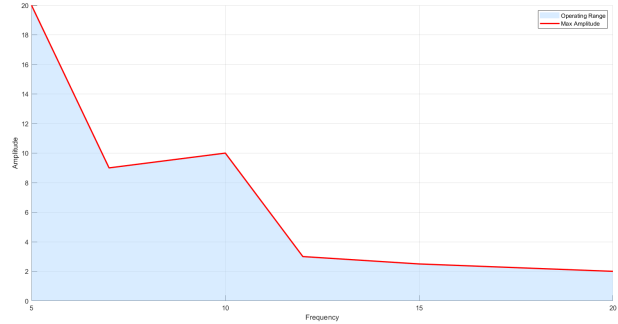
Figure 3.7: Results Moving Average extremum seeking control simulation

3.7 | Safe operating range

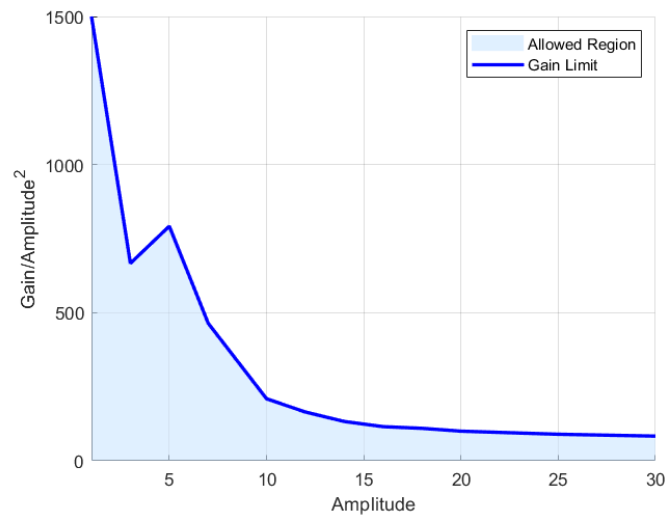
Using the simulations the safe operating ranges were found. This was done in order to identify where the system should be able to operate safely. The method used was to keep one value constant and change the other two till an unstable behavior was found. The safe operating ranges can be seen in Figure 3.8



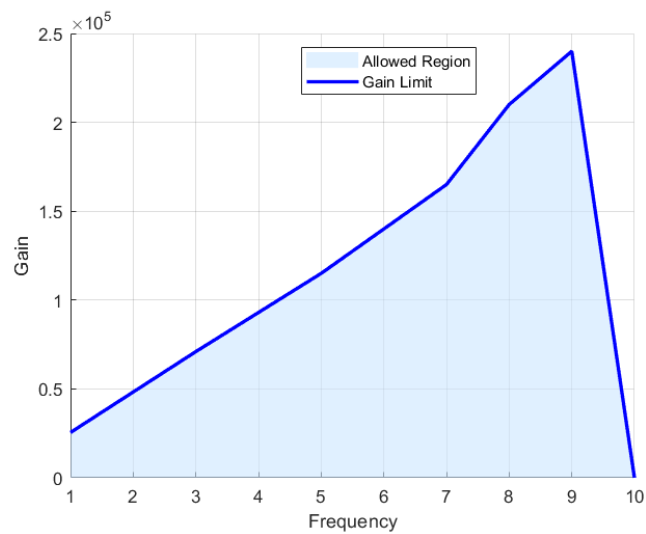
(a) Constant Gain = 50000



(b) Constant Gain = 50000 zoomed in



(c) Constant Frequency = 1



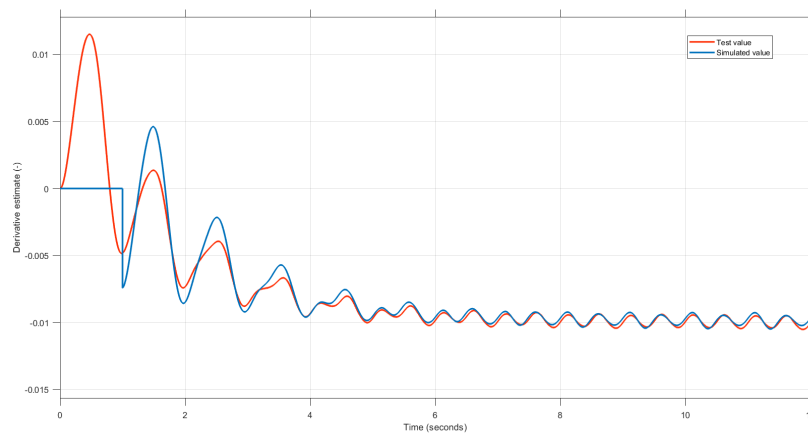
(d) Constant Amplitude = 5

Figure 3.8: Safe operating ranges

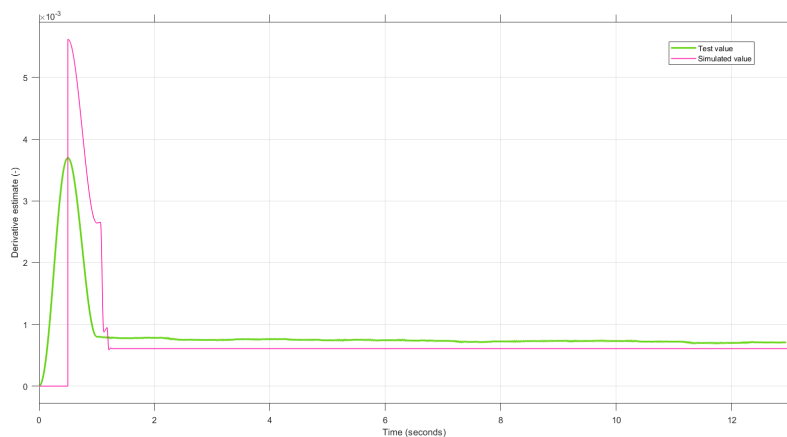
4 | Controller testing

4.1 | Derivative estimator

Before testing the full extremum seeking control loop, the derivative estimators were tested. This was done to ensure that the extremum seeking controller, would not turn unstable on the setup. This was tested by setting the gain in the optimizer to zero, ensuring the camera would oscillate around the same point. Then the system was set to oscillate and record the light measurement data. Two tests were done for both versions of the derivative estimators, one on the left of the optimum and one on the right. Then similar simulations were ran in simulink to see if the values were as expected. For these simulations the derivative estimator switch time was lowered to 0.5 seconds for higher accuracy. The results were overlaid with the simulations for comparison and can be seen in Figure 4.1.



(a) Classic comparison starting point right of optimum



(b) Moving Average comparison starting point left of optimum

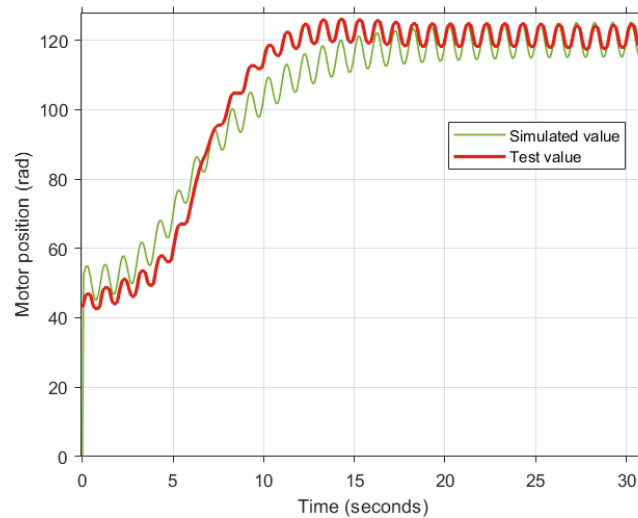
Figure 4.1: Results derivative estimator tests

As can be seen from the figures both versions worked well. They both came very close to the simulated values, which is good as the exact position on the setup is hard to know for this test. If the exact position is not the same the derivatives will also be different. So while the values are not the exact same, this test proved that the derivative estimators work as intended. The differences in both derivative estimators can also be seen from these graphs. While both derivative estimators get to a good final value, the classic derivative estimator takes a bit longer to converge to a value. As both derivative estimators were working correctly, the entire extremum seeking loop was then tested.

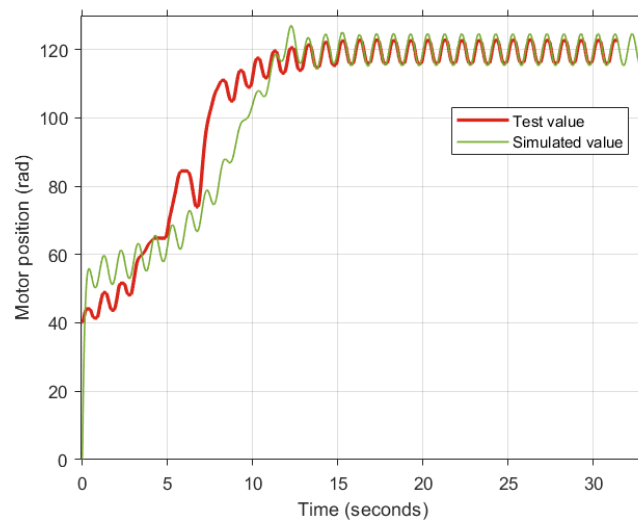
4.2 | Extremum seeking testing

Due to nature of the classic controller it keeps oscillating around the optimum. This does not appear in the moving average controller. When using the same variables, the moving average controller shows a quicker convergence, especially when considering settling time.

In order to properly compare the simulations to the real life tests, the correct starting position has to be used. Using the lookup table the optimum position can be found to be 122. Since both controllers properly converge and the position data given by the system is relative it is possible to calculate the exact starting position. This is done by subtracting the relative end position of the camera from the known optimum position. Using this starting position, the simulations were ran again so that the data could be compared properly. Then the graphs were overlaid for easy comparison, which can be seen in Figure 4.2.



(a) Classic comparison extremum from left of optimum



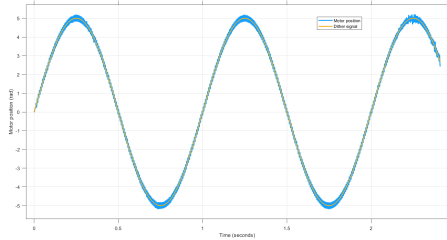
(b) Moving Average comparison extremum from left of optimum

Figure 4.2: Results extremum seeking tests

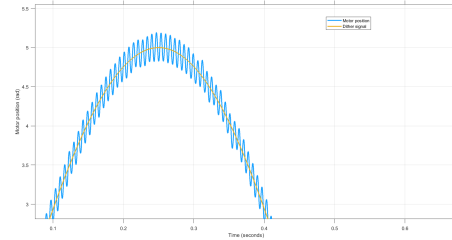
From these graphs it is clear that there is a slight difference between the real world tests and the simulations, which is expected. However from the overlaid graphs it is evident that they follow the same path when starting from the same point. This means that the simulation is close to the real world, which is good when further tuning is needed. During the testing the motion controller used in previous simulations proved to be unreliable on the setup. Sometimes the controller would instead of just controlling the sine wave from the dither signal, it would create a smaller sine wave on the existing sine wave (Figure 4.3). This is not good for the setup, so the simulations were canceled when this happened. To solve this problem

a controller with a lower bandwidth was used. Therefore the simulations for these tests were then changed to also include this edited controller. The transfer function for this controller can be seen in Equation 4.1.

$$C_b(s) = (0.005 + 0.05s) \frac{1}{6.333 * 10^{-7}s^2 + 1.137 * 10^{-3}s + 1} \quad (4.1)$$



(a) Graph of controller error



(b) Zoomed in image of problem

Figure 4.3: Problem caused by motion controller

After adding the new controller to the simulation, the main difference between the test and the simulations is the noise and friction. The camera measures the light intensity, but the setup is located in a large lab. The tests were ran while the lights were on, which means that there is a lot of ambient light. This can mess up the readings of the camera, while in the simulation a perfect function is used without noise. Friction is also not accounted for in the simulation, which can cause some differences in results.

5 | Discussion

The designing and testing of the extremum seeking controllers revealed some important points that can be improved on for the controller and simulations. Even though the controller does successfully converge to the optimum of the object function in both the simulations and the real world experiments, the differences between the simulation and real world were made very clear.

5.1 | Derivative Estimators

While the derivative estimators proved to work well in the real world, there are still some improvements to be made to the simulations. Mainly the problem that requires the derivative estimator switch could be solved. This down time, while vital for the stability causes some behaviors that are not found in the real world tests. For an accurate comparison the derivative estimator switch could be temporally added to the controller on the setup, but this would hurt overall performance if kept permanently.

Another improvement that could be made to the classic controller is to the cut off frequencies for the filters. In the entire project these were set to the standard of $\frac{\omega}{10}$, which is recommended as a solid starting point. However, this value can be tuned to fit the system, which could result in better performance over a range of operating variables.

5.2 | General simulation

The experimental results showed that while the simulations come close to the real world, they are not quite there yet. By using the lookup-table as the object function, a high degree of accuracy is achieved to the real world. However some of this accuracy is lost due to physical factors not being taken into account. Friction, noise and mechanical imperfections were not taken into account in the simulations as these are quite difficult to include in an accurate fashion. This results in differences between results such as the classic ESC oscillating around the optimum much more in the real world than in the simulations. Including these factors into the simulations could give it an extra degree of accuracy. This would be especially useful when pushing the system to be more aggressive with higher gains, frequencies and amplitudes.

5.3 | Motion controller

The controller used throughout the entire design process and simulations proved to cause some issues on the physical setup. During the experiments controller was used that is more passive with a lower bandwidth. This fixed the issue, but caused slower convergence. An improvement would be to do the motion controller process again to find a controller design which can follow higher frequencies and amplitudes, while still being able to be used in an ESC.

Overall the controllers show good potential, with both achieving the optimization goal set in the tuning process. However improvements could be made in the simulations and the motion controller to ensure higher accuracy. This higher degree of accuracy would allow for easier tuning in the future if a more aggressive ESC loop is desired.

6 | Conclusion

Two different approaches to extremum seeking control were designed and tested, in order to position a camera at the peak of a light intensity profile. These approaches were the classic and the moving average approaches. Both controllers were successfully implemented and proved to converge to the optimum in the simulations and on the real world setup.

The simulations created a strong foundation for tuning and understanding the system behavior. A gaussian function and a lookup table were used as object function implementations, which allowed for realistic simulation of the camera. However, the real world tests also showed differences between the simulations and the real world tests. Factors such as friction, noise and motion controllers had clear effects on the speed in which the controllers converged and the general stability of the system. These differences show that including more real world system behaviors into the simulation are required for improved results.

The two derivative estimator approaches were compared, and the moving average approach was found to be superior for equal parameters in convergence speed and settling time. Being less sensitive to noise and its derivative estimate not containing an oscillatory part allows it to converge and settle on the optimum very quickly.

In general, two working extremum seeking controllers were designed and tested for the physical testing setup, with one being quicker than the other. While the testing revealed some issues such as motion controllers, mechanical inaccuracies and motion controllers the existing simulations still proved quite accurate. These simulations create a strong base of which improvements can be made to include factors such as noise, friction and a different motion controller to improve simulation accuracy which leads to easier controller tuning in the future.

7 | References

A. Varnavides. Preparation of experimental setup. Bachelor's thesis, Technische Universiteit Eindhoven, 2025.

A | Appendix A



Figure A.1: E/box used



Figure A.2: Raspberry Pi used

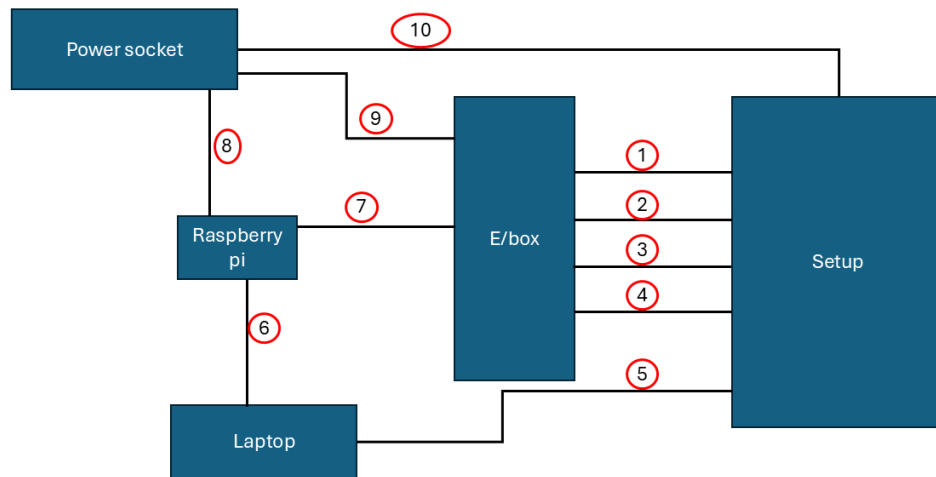


Figure A.3: Wiring graphic

Number	Port 1	Port 2
1	Detector output (Setup)	Analog in (1) (E/box)
2	Motor voltage input (2,5 V in) (Setup)	Analog out (1) (E/box)
3	Encoder 2 (Setup)	Enc/PWM (1) (E/box)
4	Attached to setup (Between 1 & 2) (Setup)	Digital in (Specific pins see image E/box)
5	Attached to setup (Between 1 & 2) (Setup)	Laptop USB-A port
6	Laptop USB-A port	Raspberry Pi USB-C (bottom)
7	Raspberry Pi Ethernet port	Ethercat input E/box (Same side as power)
8	Raspberry Pi USB-C (top)	Wall socket
9	Power port (E/box)	Wall socket
10	Setup power port	Wall socket

Connection 4 should use the left most pins on the E/box for the Left light gate, those next to it for the middle light gate, and the most right pins for the right light gate.

B | Manual building an Extremum Seeking Controller

B.1 | System design

First the general system design will be discussed. There are three main parts to the simulation, that being the position controller, the derivative estimator and the optimizer. Additionally the dither signal and the object function need to be added. For the dither signal a sine wave block is used. The objective function also needs to be included, which can be done in different ways which will be discussed in B.1.5. A general overview of the simulation can be seen in Figure B.1.

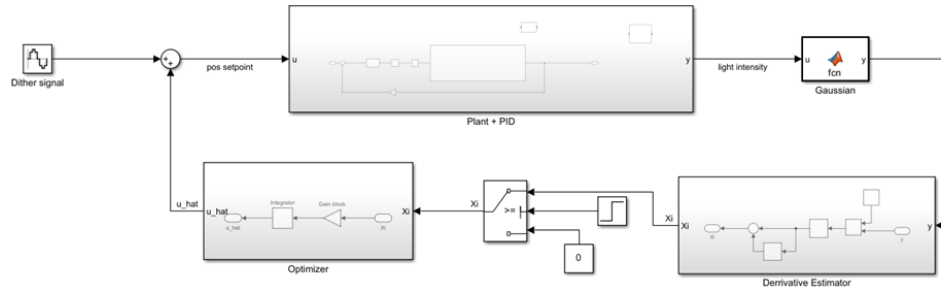


Figure B.1: General ESC control loop

B.1.1 | Motion controller

The motion controller consists of a simple feedback controller. The design of a position controller will not be discussed in this manual. An example transfer function for the controller can be seen in Equation B.1 and the layout of the motion controller can be seen in Figure B.2.

$$C_b(s) = (0.005 + 0.05s) \frac{1}{6.333 \cdot 10^{-7}s^2 + 1.137 \cdot 10^{-3}s + 1} \quad (\text{B.1})$$

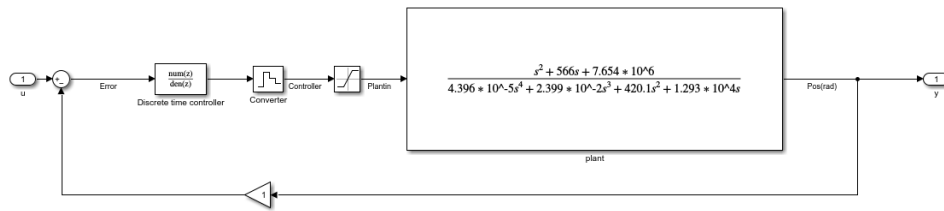


Figure B.2: Layout of the motion controller

For more realism, a saturation block can be added between the controller and the plant. This way the real life voltage limiting can be simulated.

B.1.2 | Derivative estimator

There are two options for the derivative estimator. The first uses a high pass, a low pass filter, and a demodulation signal to filter out the unwanted components of the signal. The second uses a moving average window and the same demodulation signal to eliminate the unwanted components and find the derivative.

B.1.2.1 Classic Extremum seeking control

For the classic version the derivative estimator would look like figure B.3. For the filter cutoff values the rule of thumb of 0.1 times the dither frequency can be used. The demodulation signal is the same as the dither signal with the amplitude changed to $2/a$ instead of a .

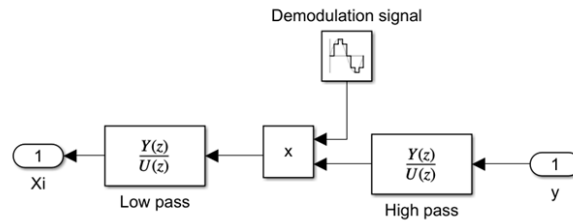


Figure B.3: Classic derivative estimator layout

B1.2.2 Moving average

To create a moving average filter, several blocks are needed in Simulink. Use an integrator block into a delay and sum block to create the moving average yourself. Then add a gain block after the sum block. The demodulation signal can be the same as before. The delay block should have the amount of samples in a single perturbation and the gain block should be the same as the dither frequency in Hz. The layout can be seen in Figure B.4.

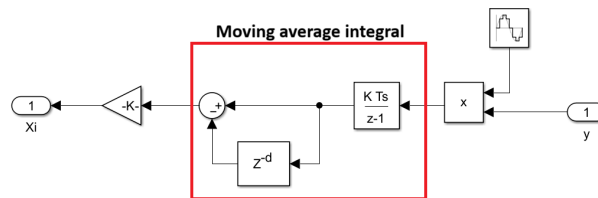


Figure B.4: Moving Average derivative estimator

B.1.3 | Optimizer

This section consists of a gain block that leads into an integrator block. This gain block is the one of which the value can be changed for tuning. The layout for this can be seen in Figure B.5

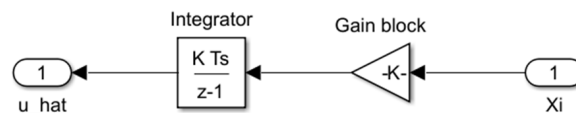


Figure B.5: Optimizer layout

B.1.4 | Derivative Estimator Switch

Some other small parts are needed in the simulation to make it work well and make it more realistic. The first of which being the derivative estimator switch. When starting the simulation the model forces the position to go to whatever location is selected. However when doing this jump the derivative estimator cannot return an accurate value and interferes with the converging process. To prevent this from happening, the value from the derivative estimator is set to zero for the first couple seconds. In order to make this happen the following setup can be used. The step block has a step time which can be changed, this will be the time that the derivative estimator is zero. The block in the middle of the setup is a switch block. The layout can be seen in Figure B.6.

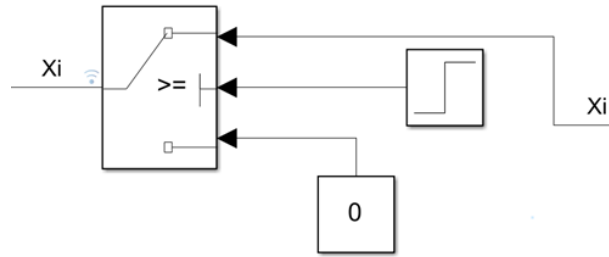


Figure B.6: Switch blocks

B.1.5 | Objective function

There are several different methods to include the objective function, two of which will be discussed in this section.

B.1.5.1 Gaussian function

The first method for including the objective function is creating a gaussian function. This has the advantage of being a smooth function, which causes less errors than the next option, but is less accurate. A gaussian function has the shape of Equation B.2.

$$y = a * e^{\left(\frac{-(x-b)^2}{c}\right)} \quad (\text{B.2})$$

Here b is the x coordinate of the peak, a is the height of the peak and c can be used to adjust the width of the graph.

B.1.5.2 Lookup table

For a more accurate implementation than the gaussian function, a lookup table can be used. To use this a measurement of the physical objective function is needed. Then perform a fit function on the data with fitType 'smoothing spline'. This will then create the data points in 2 variables, x and y. These can then be pulled from the workspace to Simulink using a 2-D lookup table. This will give a more accurate object function, but will create problems when using very low amplitudes ($a \ll 1$).

B.2 | System Analysis

Before creating an ESC controller it is important to see what the system can handle. For example, if a dither signal is chosen with a frequency which is too high, the system cannot handle it and it will fail for whatever variables you choose.

B.2.1 | What frequency and amplitude can the system handle

Inject a sinusoidal signal into the open loop of the system, i.e. the gain in the optimizer section set to zero and see how the system responds. If the system has a large phase lag on the dither, lower the frequency. If the system's response shows larger amplitudes than the dither, lower the amplitude of the dither signal. These values determine the baseline for the maximum values of the frequency and amplitudes.

B.2.2 | See what frequency the Derivative Estimator can handle

Choose an objective function with a known derivative (x^2 for example). Now change the dither signal to add a bias where the derivative is known (set the initial position to 0!). Then set the gain in the integrator to zero and keep increasing the frequency. See what the maximum frequency and amplitude are where the derivative estimate is still accurate to the known value.

B.3 | Choosing ESC variables

The last step is to choose the ESC variables. These variables will be split up into two parts, the first are constant values and the second will be the tunable variables.

B.3.1 | Constant variables

The following variables will be constant and system dependent:

- The derivative estimator filter cutoffs should be set to 0.1 times the dither frequency.
- The moving average derivative estimator delay block should be $\frac{1}{f*dt}$ where dt is the sampling time and f the dither frequency.
- The gain block in the moving average derivative estimator should be equal to the dither frequency in Hz.
- Sampling time should be set to the same as the system's (in seconds). This should be put into every block that requires sample times.
- Saturation block should be set to the values that the physical system limits the voltage at. For this report it was (2.5, -2.5).

B.3.2 | Tunable variables

Firstly the starting position for the simulation can be chosen by entering a value into the initial position entry in the integrator block in the optimizer subsystem. For the actual tuning there are three main values to tune. Those being the dither amplitude, dither frequency and the controller gain. For all three of these values it is the goal to make them as large as possible without making the system unstable. The dither amplitude and frequency are quite self explanatory. The higher frequency means that the system has more measurements to compare which means that it can find the optimum quicker, but a frequency that is too high runs the risk of the motion controller not being able to keep up which makes the system unstable. A higher amplitude means that it takes larger steps which makes it easier for the derivative estimator to find the gradient. This results in a quicker convergence to the optimum. However if the amplitude gets too large it will result in the system overshooting the optimum which leads to irregularities from the derivative estimator, which leads to an unstable system. The same logic applies for the controller gain, this is the gain by which the derivative estimate is multiplied in the optimizer section. A higher gain means that the system moves farther in the estimated direction, but if this gain is too high it will overshoot once again resulting in an unstable system. A good tuning method is to choose a frequency to start at and only tune the other two variables until you are satisfied with the convergence time. Then start slowly increasing the frequency until you find a convergence time which satisfies the conditions of the setup.

B.4 | Common errors

B.4.1 | Trouble connecting to Raspberry Pi

On the current setup a Raspberry Pi model 4b is used, which is automatically selected in Matlab 2023b. When using a newer model Raspberry Pi or a different version of Matlab, check in the add on manager for the Matlab support package for Raspberry Pi. There you can see what board is pre-selected, to change the Raspberry Pi model a ethernet cable could be needed.

B.4.2 | Missing monitor and tune button

In order to run the tests on the setup, the button monitor and tune is pressed. This is listed under the menu hardware. If this is missing follow these steps:

1. Press ctrl+e while in your Simulink file
2. Go to hardware implementation
3. Select the 'Raspberry Pi' under hardware board

B.4.3 | When faced with an error about inline s functions, check the following

1. Press ctrl+e while in your simulink file
2. Go to code generation → Advanced parameters (this is hidden at the bottom of the menu) (See figure B.7)
3. Replace the 'Post code generation command' with this: `if (verLessThan("matlab", "9.11")) codertarget.postCodeGenHookCommand(h); SPERTE_postGenFunc(modelName, buildInfo); else; SPERTE_postGenFunc(h, buildInfo); end`

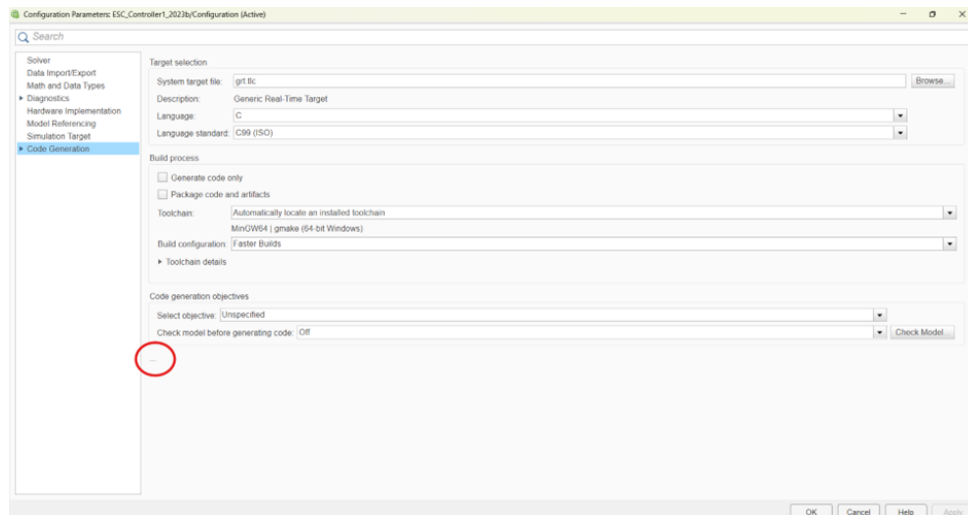


Figure B.7: Advanced parameters location

B.4.4 | Setup making loud noise when running model

This could be due to a controller that is too aggressive, try using a controller with a lower bandwidth to see if this solves the issue.

For more errors concerning connection to the Raspberry Pi, see [Varnavides \(2025\)](#)